

Summary

This briefing explain critical concepts in **Database Management Systems (DBMS)** related to **transaction management, concurrency control, and recoverability**. Modern databases must balance **performance** and **consistency**. While serial schedules guarantee correctness, they are inefficient. Non-serial (concurrent) schedules improve performance but introduce risks such as inconsistency and data anomalies.

The document explains mechanisms that ensure correctness in concurrent execution, including **serializability determination using topological sorting, recoverability classifications, and lock-based concurrency control protocols**. It also highlights exam-oriented insights relevant for **GATE 2026** preparation.

Key Takeaways

- **Recoverability hierarchy** (Recoverable → Cascadeless → Strict) determines how safely failures can be handled.
- **Topological sorting** provides the serial order of transactions in conflict-serializable schedules.
- **Lock-based protocols** (Shared and Exclusive locks) are the practical tools used to prevent anomalies such as Dirty Reads and Lost Updates.

Academic Strategy and Preparation for GATE 2026

A strategic, weightage-driven approach is essential for competitive examinations.

Subject Prioritization

- Focus first on **General Aptitude** and **Engineering Mathematics**, as they carry high and predictable weightage.

Balanced Learning Approach

- Use **live classes** to maintain discipline and continuity.
- Use **recorded sessions** for lengthy or concept-heavy subjects such as **COA, C Programming, and Data Structures**.

Integrated Practice

- Avoid postponing practice.
- Solve **PYQs immediately after completing each topic** to reinforce conceptual clarity and exam readiness.

Serializability and Topological Sorting

Serializability is the correctness criterion for concurrent schedules. A schedule is correct if its outcome matches that of some serial execution of the same transactions.

Types of Serializability

Conflict Serializability

- Based on conflicting operations.
- If a schedule is conflict serializable, it is automatically view serializable.

View Serializability

- Broader condition.
- A schedule that fails conflict serializability may still be view serializable.

Topological Sorting Method

Topological sorting is used to determine the **serializability order** from a **precedence graph**.

Algorithmic Steps

1. **Identify In-Degrees**
Count incoming edges for each transaction node.
2. **Select Zero In-Degree Vertices**
Choose transactions with no incoming edges as valid starting points.
3. **Remove and Repeat**
Remove the selected transaction and its outgoing edges. Recompute in-degrees and repeat until all transactions are ordered.

Interpretation Scenarios

- **Simple Chain ($T1 \rightarrow T2 \rightarrow T3$)**
Serial order: T1, T2, T3.
- **Multiple Zero In-Degree Vertices**
Multiple valid serial orders may exist, all of which are correct.

Problems Arising from Concurrent Execution

If concurrency is not properly controlled, the following anomalies may occur:

Dirty Read (Write–Read / WR)

- A transaction reads data written by another transaction that has not yet committed.
- If the writer rolls back, the reader has used invalid data.

Unrepeatable Read (Read–Write / RW)

- A transaction reads the same data item twice but gets different values.
- Occurs because another transaction updates and commits the data in between.

Lost Update (Write–Write / WW)

- Two transactions update the same data item.
- One update overwrites the other without awareness.
- Commonly caused by **Blind Writes** (writing without reading first).

Recoverability and Atomicity

Recoverability ensures that the database can be restored to a consistent state after transaction failure, thereby enforcing **Atomicity**.

Classification of Schedules

Recoverable Schedule

- If T_j reads data written by T_i , then T_i must commit before T_j .
- Guarantees recoverability but may still allow Dirty Reads.

Cascadeless Schedule

- A transaction can read data only after the writing transaction commits.
- Prevents cascading rollbacks.

Strict Schedule

- A transaction cannot read or write a data item until the last transaction that wrote it has committed.
- Most restrictive and safest.
- Prevents Dirty Reads, Unrepeatable Reads, and Lost Updates.

Lock-Based Protocols

Locks are the primary mechanism for enforcing concurrency control in DBMS.

1. Lock Types

Shared Lock (S)

- Used for Read operations.
- Multiple transactions can hold shared locks on the same data item simultaneously.

Exclusive Lock (X)

- Used for Write (or Read-Write) operations.
- Only one transaction can hold an exclusive lock.
- Blocks both shared and exclusive locks by other transactions.

2. Implementation Rules

Data Item Dependency

- Locks are applied per data item.
- Transactions can lock different items simultaneously.
- If a transaction holds a lock on item A, other transactions must wait to lock A until it is unlocked.

Wait Conditions

- If a requested lock conflicts with an existing lock, the requesting transaction must wait.

3. Execution Flow

The standard execution protocol follows:

1. **Lock Acquisition**
Transaction requests S or X lock.
2. **Operation Execution**
Perform Read or Write.
3. **Unlock**
Release the lock so other waiting transactions can proceed.

